

Тема 1.6 Циклы while и for. Исключения

Оператор цикла for

Предположим, нужно вывести все числа от 1 до 100 по одному на строке. Обычным способом пришлось бы писать 100 строк кода:

```
print (1)
print (2)
. . . .
print (100)
```

При помощи циклов то же действие можно выполнить одной строкой кода:

```
for x in range (1, 101) : print (x)
```

Иными словами, циклы позволяют выполнить одни и те же инструкции многократно.

Цикл for применяется для перебора элементов последовательности и имеет такой формат:

```
for <Текущий элемент> in <Последовательность>:
    <Инструкции внутри цикла>
[else:
    <Блок, выполняемый, если не использовался оператор break>
]
```

Здесь присутствуют следующие конструкции:

- **<Последовательность>** — объект, поддерживающий механизм итерации. Например: строка, список, кортеж, диапазон, словарь и др.;

- **<Текущий элемент>** — на каждой итерации через этот параметр доступен текущий элемент последовательности или ключ словаря;

- **<Инструкции внутри цикла>** — блок, который будет многократно выполняться;

- если внутри цикла не использовался оператор **break**, то после завершения выполнения цикла будет выполнен блок в инструкции **else**. Этот блок не является обязательным.

Пример перебора букв в слове:

```
# -*- coding: utf-8 -*-
for s in "str":
    print (s, end=" ")
else:
    print ("\nЦикл выполнен")
```

Результат:

s t r

Цикл выполнен

Теперь выведем каждый элемент списка и кортежа на отдельной строке:

```
for x in [1, 2, 3]:
    print(x)
for y in (1, 2, 3):
    print(y)
```

Результат:

```
1
2
3
1
2
3
```

Цикл **for** позволяет также перебрать элементы словарей, хотя словари и не являются последовательностями. В качестве примера выведем элементы словаря двумя способами. Первый способ использует метод **keys ()**, возвращающий объект **dict_keys**, который содержит все ключи словаря. Во втором способе просто указывается словарь в качестве параметра — на каждой итерации цикла будет возвращаться ключ, с помощью которого внутри цикла можно получить значение, соответствующее этому ключу.

```
arr = {"x": 1, "y": 2, "z": 3}
```

```
print(arr.keys())
```

Результат:

```
dict_keys(['x', 'y', 'z'])
```

```
arr = {"x": 1, "y": 2, "z": 3}
```

```
for key in arr.keys(): # Использование метода keys()
```

```
    print(key, arr[key])
```

Результат:

```
x 1
```

```
y 2
```

```
z 3
```

```
for key in arr: # Словари также поддерживают итерации
```

```
    print(key, arr[key])
```

Результат:

```
x 1
```

```
y 2
```

```
z 3
```

Обратите внимание на то, что элементы словаря выводятся в произвольном порядке, а не в порядке, в котором они были указаны при создании объекта. Чтобы вывести элементы в алфавитном порядке, следует отсортировать ключи с помощью функции **sorted ()**:

```
arr = {"x": 1, "y": 2, "z": 3}
```

```
for key in sorted(arr):
```

```
    print(key, arr[key])
```

Результат:

```
x 1
```

```
y 2
```

```
z 3
```

С помощью цикла **for** можно перебирать сложные структуры данных. В качестве примера выведем элементы списка кортежей.

```
arr = [(1, 2), (3, 4)]
```

```
for a, b in arr:
```

```
print(a, b)
```

Результат:

```
1 2
```

```
3 4
```

Оператор цикла **while**

Выполнение инструкций в цикле **while** продолжается до тех пор, пока логическое выражение истинно. Цикл **while** имеет следующий формат:

```
<Начальное значение>
```

```
while <Условие>:
```

```
    <Инструкции>
```

```
    <Приращение>
```

```
[else:
```

```
    <Блок, выполняемый, если не использовался оператор break>
```

```
]
```

Последовательность работы цикла **while**:

- переменной-счетчику присваивается начальное значение;
- проверяется условие и, если оно истинно, выполняются инструкции внутри цикла, иначе выполнение цикла завершается;
- переменная-счетчик изменяется на величину, указанную в параметре **<Приращение>**;
- переход к пункту 2;
- если внутри цикла не использовался оператор **break**, то после завершения выполнения цикла будет выполнен блок в инструкции **else**. Этот блок не является обязательным.

Выведем все числа от 1 до 10, используя цикл **while**.

```
i = 1    # <Начальное значение>
```

```
while i < 11: # <Условие>
```

```
    print(i) # <Инструкции>
```

```
    i += 1  # <Приращение>
```

Выведем все числа от 100 до 1:

```
i = 100
```

```
while i:
```

```
    print(i)
```

```
    i -= 1
```

Обратите внимание на условие - оно не содержит операторов сравнения. На каждой итерации цикла необходимо вычитать единицу из значения переменной-счетчика. Как только значение будет равно 0, цикл остановится. Число 0 в логическом контексте эквивалентно значению **False**, а проверка на равенство выражения значению **True** выполняется по умолчанию.

С помощью цикла **while** можно перебирать и элементы различных структур. Но в этом случае следует помнить, что цикл **while** работает медленнее цикла **for**. В качестве примера умножим каждый элемент списка на 2:

```
arr = [1, 2, 3]
```

```
i, count = 0, len(arr)
while i < count:
    arr[i] *= 2
    i += 1
print(arr)
```

Оператор цикла `continue`

Оператор **`continue`** позволяет перейти к следующей итерации цикла до завершения выполнения всех инструкций внутри цикла. В качестве примера выведем все числа от 1 до 20, кроме чисел от 5 до 15 включительно.

```
for i in range(1, 21):
    if 4 < i < 16:
        continue # Переходим на следующую итерацию цикла
    print(i)
```

Оператор цикла `break`

Оператор **`break`** позволяет прервать выполнение цикла досрочно. Для примера выведем все числа от 1 до 10 еще одним способом.

```
i = 1
while True:
    if i > 10: break # Прерываем цикл
    print(i)
    i += 1
```

Здесь в условии указано значение **`True`**. В этом случае выражения внутри цикла станут выполняться бесконечно. Однако использование оператора **`break`** прерывает выполнение цикла, как только будет напечатано число 10.

Цикл **`while`** совместно с оператором **`break`** удобно использовать для получения неопределенного заранее количества данных от пользователя. В качестве примера просуммируем неопределенное количество чисел.

```
# -*- coding: utf-8 -*-
print("Введите слово 'stop' для получения результата")
summa = 0
while True:
    x = input("Введите число: ")
    if x == "stop":
        break # Выход из цикла
    x = int(x) # Преобразуем число в строку
    summa += x
print("Сумма чисел равна:", summa)
input()
```

Функции `range()` и `enumerate()`

До сих пор необходимо было выводить элементы последовательностей. Теперь попробуем умножить каждый элемент списка на 2:

```
arr = [1, 2, 3]
for i in arr:
    i = i * 2
print(arr)
```

Как видно из примера, список не изменился. Переменная **i** на каждой итерации цикла содержит лишь копию значения текущего элемента списка. Изменить таким способом элементы списка нельзя. Чтобы получить доступ к каждому элементу, можно, например, воспользоваться функцией **range ()** для генерации индексов. Функция **range()** имеет следующий формат:

```
range([<Начало>, ]<Конец>[, <Шаг>])
```

Первый параметр задает начальное значение. Если параметр **<Начало>** не указан, то по умолчанию используется значение 0. Во втором параметре указывается конечное значение. Следует заметить, что это значение не входит в возвращаемые значения. Если параметр **<Шаг>** не указан, то используется значение 1. Функция возвращает диапазон - особый объект, поддерживающий итерационный протокол. С помощью диапазона внутри цикла **for** можно получить значение текущего элемента. В качестве примера умножим каждый элемент списка на 2.

```
arr = [1, 2, 3]
for i in range(len(arr)):
    arr[i] *= 2
print(arr)
```

В этом примере получаем количество элементов списка с помощью функции **len ()** и передаем результат в функцию **range ()**. В итоге функция **range()** возвращает диапазон значений от 0 до **len (arr)** - 1. На каждой итерации цикла через переменную **i** доступен текущий элемент из диапазона индексов. Чтобы получить доступ к элементу списка, указываем индекс внутри квадратных скобок. Умножаем каждый элемент списка на 2, а затем выводим результат с помощью функции **print ()**.

Рассмотрим несколько примеров использования функции **range()**. Выведем числа от 1 до 100:

```
for i in range(1, 101): print(i)
```

Можно не только увеличивать значение, но и уменьшать его. Выведем все числа от 100 до 1:

```
for i in range(100, 0, -1): print(i)
```

Можно также изменять значение не только на единицу. Выведем все четные числа от 1 до 100:

```
for i in range(2, 101, 2): print(i)
```

В **Python 3** поведение функция **range()** возвращает диапазон. Чтобы получить список чисел, следует передать диапазон, возвращенный функцией **range()**, в функцию **list()**.

```
obj = range(len([1, 2, 3]))
print(obj)
```

Результат:

```
range(0, 3)
```

```
print(obj[0], obj[1], obj[2]) # Доступ по индексу
```

Результат:

```
0 1 2
```

```
print(obj[0:2]) # Получение среза
```

Результат:

```
range(0, 2)
```

```
i = iter(obj)
```

```
print(next(i), next(i), next(i)) # Доступ с помощью итераторов
```

Результат:

```
0 1 2
```

```
print(list(obj)) # Преобразование диапазона в список
```

Результат:

```
[0, 1, 2]
```

```
print(1 in obj, 7 in obj) # Проверка вхождения значения
```

Результат:

```
(True, False)
```

Диапазон поддерживает два полезных метода:

- **index (<Значение>)** - возвращает индекс элемента, имеющего указанное значение. Если значение не входит в диапазон, возбуждается исключение **ValueError**. Пример:

```
obj = range(1, 5)
```

```
print(obj.index(1), obj.index(4))
```

Результат:

```
0 3
```

```
print(obj.index(5))
```

Результат:

```
Traceback (most recent call last):
```

```
File "<pyshell#11>", line 1, in <module>
```

```
obj.index(5)
```

```
ValueError: 5 is not in range
```

- **count (<Значение>)** - возвращает количество элементов с указанным значением. Если элемент не входит в диапазон, возвращается значение 0. Пример:

```
obj = range(1, 5)
```

```
print(obj.count(1), obj.count(10))
```

Результат:

```
1 0
```

Функция **enumerate (<объект>[, start=0])** на каждой итерации цикла **for** возвращает кортеж из индекса и значения текущего элемента. С помощью необязательного параметра **start** можно задать начальное значение индекса. В качестве примера умножим на 2 каждый элемент списка, который содержит четное число.

```
arr = [1, 2, 3, 4, 5, 6]
```

```
for i, elem in enumerate(arr):
```

```

if elem % 2 == 0:
    arr[i] *= 2
print(arr)

```

Результат:

```
[1, 4, 3, 8, 5, 12]
```

Функция **enumerate()** не создает список, а возвращает итератор. С помощью функции **next()** можно обойти всю последовательность. Когда перебор будет закончен, возбуждается исключение **StopIteration**:

```

arr = [1, 2]
obj = enumerate(arr, start=2)
print(next(obj))

```

Результат:

```
(2, 1)
```

```
print(next(obj))
```

Результат:

```
(3, 2)
```

```
print(next(obj))
```

Результат:

Traceback (most recent call last):

File "<pyshell#20>", line 1, in <module>

next(obj)

StopIteration

Цикл **for** при работе активно использует итераторы, но делает это незаметно.

Пример решения задачи на языке программирования Python

Задача. Последовательно вводится десять целых чисел. Найдите произведение всех положительных и всех отрицательных чисел, предварительно вычислив их сумму.

Разработка алгоритма решения задачи представлена на рисунке 1.

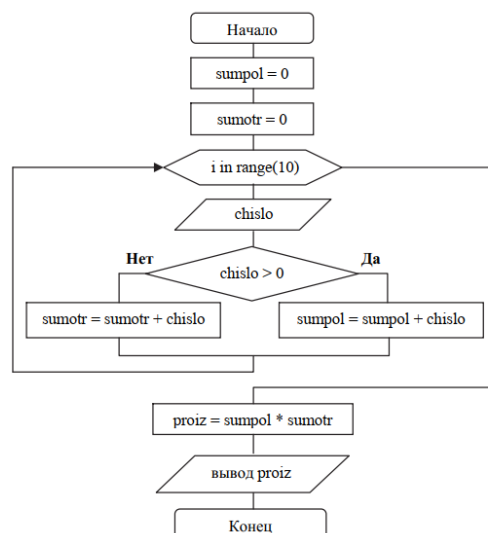


Рисунок 1 – Алгоритм решения задачи

Решение задачи:

```
sumotr=0
sumpol=0
for i in range(10):
    chislo=int(input("Введите число "))
    if chislo>0:
        sumpol=sumpol+chislo
    else:
        sumotr=sumotr+chislo
proiz=sumotr*sumpol
print("Произведение положительных и отрицательных чисел = ",proiz)
```